

Vibe Engineering — QuipuNet Client Mode: IP de Origen + Notificaciones Bidireccionales

Estado:  E2E IMPLEMENTADO — Fases 1-4 + Fase 9 completas, build verificado 0 errores

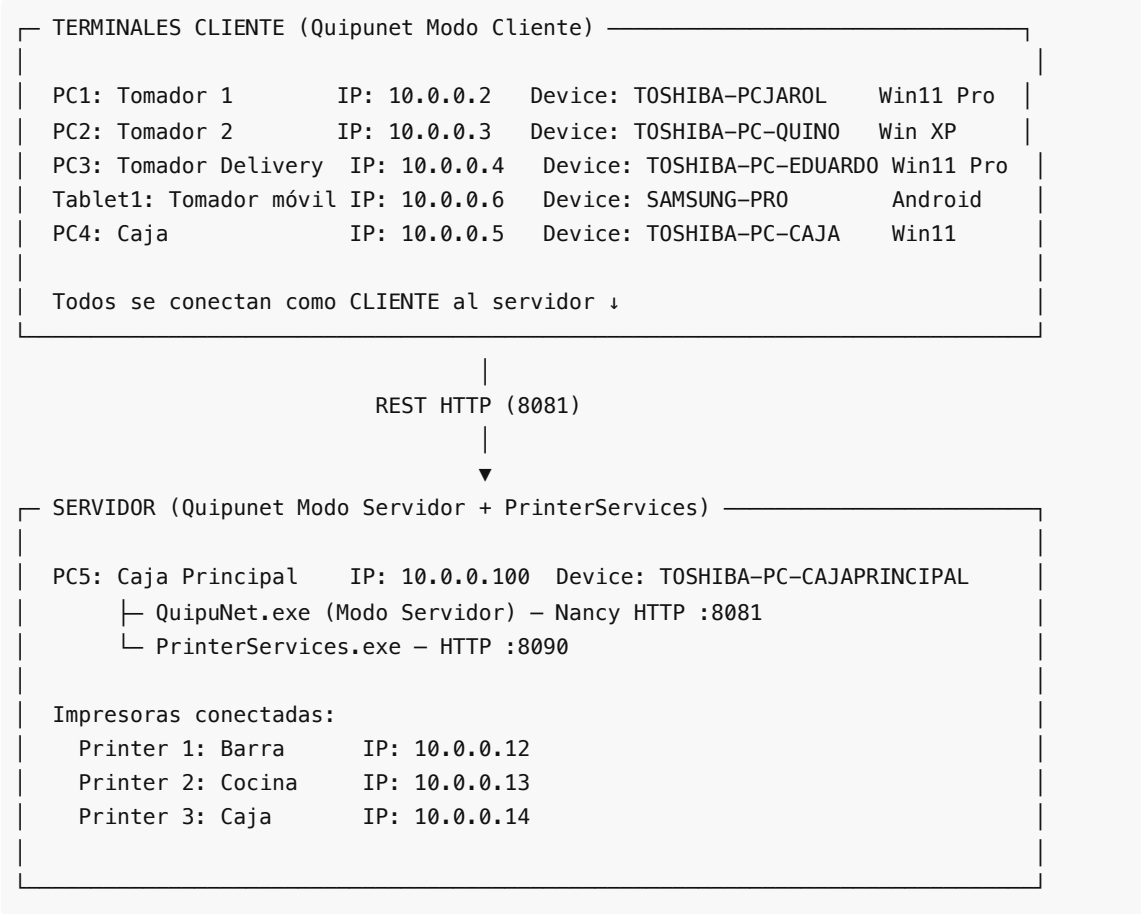
Fecha: 2026-03-08

Objetivo: Que PrinterServices identifique qué terminal cliente originó cada impresión (IP, DeviceName) y notifique tanto al servidor como al cliente real del resultado.

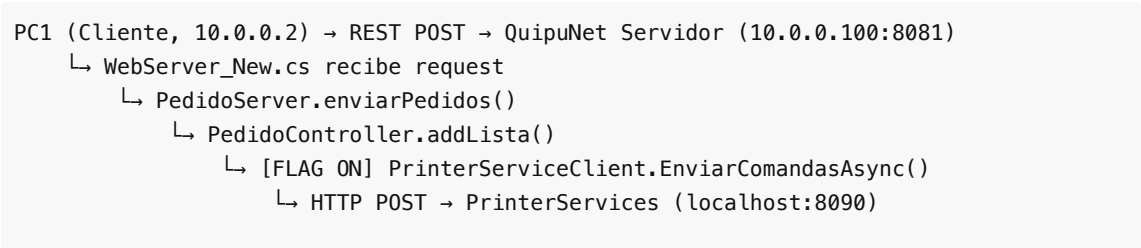
Prerrequisito: vibe_engineering_acoplequipunet.md (Fases 7A, 7B, 23 implementadas)

1. Contexto: Arquitectura Cliente/Servidor

1.1 Topología de red típica de un restaurante



1.2 Flujo actual de impresión (cliente → servidor → PrinterServices)



```
↳ PrintController.PostComandas()  
    ↳ Enqueue(job) → PrintWorker → TCP:9100 → Impresora
```

1.3 Problema actual

PrinterServiceClient.EnriquecerImpresion() (QuipuNetX) siempre envía:

```
json["ip_origen"] = Util.getIpServerPermanente() ?? ""; // Siempre 10.0.0.100  
json["device_id_origen"] = Util.DeviceName ?? "";      // Siempre TOSHIBA-PC-  
CAJAPRINCIPAL
```

Consecuencia: PrinterServices cree que TODAS las impresiones vienen del servidor. Cuando intenta notificar el resultado (DONE/FAILED/EXPIRED), solo notifica a **10.0.0.100**. La terminal cliente (PC1 = 10.0.0.2) que originó la comanda **nunca se entera** del resultado.

1.4 Lo que queremos lograr

```
PC1 (10.0.0.2) envía pedido → QuipuNet Servidor intercepta IP = 10.0.0.2  
    → PrinterServiceClient envía ip_origen = 10.0.0.2, ip_servidor = 10.0.0.100  
        → PrinterServices guarda ambas IPs en print_jobs  
            → Cuando imprime (DONE/FAILED):  
                1. Notifica a 10.0.0.100 (servidor) ← SIEMPRE  
                2. Notifica a 10.0.0.2 (cliente)   ← SOLO si es diferente al servidor
```

Resultado: Tanto el servidor como el cliente saben el resultado de la impresión.

2. Hallazgo clave: GetClientIp() ya existe

En QuipuNetX/ws/WebServer_New.cs líneas 4850-4895 ya existe:

```
private static string GetClientIp(Request request)  
{  
    // Busca en orden: X-Forwarded-For → X-Real-IP → CF-Connecting-IP →  
    UserHostAddress  
    // Retorna la IP real del cliente que hizo el HTTP request  
}
```

Ya se usa en al menos un endpoint (línea 4589: cobrarCredibancoResponse).

3. Plan de Implementación

Fase 1: Propagar ipClienteRest desde WebServer → Controller

Objetivo: Interceptar la IP del cliente REST en los endpoints de impresión.

3.1.1 Modificar endpoints en WebServer_New.cs (SimpleModule)

Archivo: QuipuNetX/ws/WebServer_New.cs

Endpoints que disparan impresión y necesitan pasar ipClienteRest :

Endpoint	Línea aprox	Llama a
POST /api/rest/pedido/enviarPedidos	~3628	PedidoServer.enviarPedidos()
POST /api/rest/pedido/enviarPedidosMercado	~3674	PedidoServer.enviarPedidosMercado()
GET /api/rest/pedido/reImprimirPedido	~579	PedidoServer.reImprimirPedido()
Endpoints de cobro/venta	varios	OrdenpedidoServer.* / VentaController.*

Patrón de cambio (ejemplo `enviarPedidos`):

```
// ANTES:
Post("/api/rest/pedido/enviarPedidos", paramsGet =>
{
    // ...
    string datos = PedidoServer.enviarPedidos(data, paramsGet["es_movil"],
paramsGet["compress"]);
    // ...
});

// DESPUÉS:
Post("/api/rest/pedido/enviarPedidos", paramsGet =>
{
    // ...
    var ipClienteRest = GetClientIp(this.Request) ?? ""; // ← IP del cliente que
hace el REST
    string datos = PedidoServer.enviarPedidos(data, paramsGet["es_movil"],
paramsGet["compress"], ipClienteRest);
    // ...
});
```

3.1.2 Modificar endpoints en `PedidoModule.cs` (Nancy Module)

Archivo: `QuipuNetX/ws/modules/PedidoModule.cs`

Mismos endpoints pero en el módulo Nancy. `PedidoModule` hereda de `CorsModuleBase` y tiene acceso a `this.Request.UserHostAddress`.

Patrón: Usar `this.Request.UserHostAddress` directamente o copiar el helper `GetClientIp` al `BaseModule`.

3.1.3 Modificar `PedidoServer`

Archivo: `QuipuNetX/server/PedidoServer.cs`

Agregar parámetro opcional `string ipClienteRest = ""` a los métodos que terminan en `Controllers` de impresión:

```
// ANTES:
public static string enviarPedidos(string data, string esMovil, string compress)

// DESPUÉS:
```

```
public static string enviarPedidos(string data, string esMovil, string compress,
string ipClienteRest = "")
```

Propagar a `PedidoController.addLista()` .

3.1.4 Modificar `PedidoController.addLista()`

Archivo: `QuipuNetX/controller/PedidoController.cs`

Agregar parámetro opcional y pasarlo a `PrinterServiceClient` :

```
// ANTES (línea ~612):
respuestaPS = await PrinterServiceClient.Instance.EnviarComandasAsync(
    comandasParaImprimir, pedidoIds, "Imprimiendo comandas...");

// DESPUÉS:
respuestaPS = await PrinterServiceClient.Instance.EnviarComandasAsync(
    comandasParaImprimir, pedidoIds, "Imprimiendo comandas...", ipClienteRest);
```

3.1.5 Mismo patrón para `VentaController`

Archivo: `QuipuNetX/controller/VentaController.cs`

Los métodos `addVentaRapidaMovil` y `addVentaDeliveryMovil` usan `ServerImpresionService.Instance.ImprimirAsync(ctx)` . El `ImpresionContext` necesita un campo `IpClienteRest` que se propague hasta `PrinterServiceClient` .

Fase 2: Propagar `ipClienteRest` hasta `PrinterServiceClient`

3.2.1 Modificar `EnviarComandasAsync()`

Archivo: `QuipuNetX/Services/Print/PrinterServiceClient.cs`

```
// ANTES:
public async Task<Respuesta> EnviarComandasAsync(
    IList<Impresion> impresionList,
    IList<string> pedidoIds = null,
    string mensajeDinamico = "Imprimiendo comandas...")

// DESPUÉS:
public async Task<Respuesta> EnviarComandasAsync(
    IList<Impresion> impresionList,
    IList<string> pedidoIds = null,
    string mensajeDinamico = "Imprimiendo comandas...",
    string ipClienteRest = "") // ← NUEVO
```

3.2.2 Modificar `EnriquecerImpresion()`

Archivo: `QuipuNetX/Services/Print/PrinterServiceClient.cs`

Agregar parámetro `ipClienteRest` y cambiar lógica de `ip_origen` :

```
// ANTES (línea 262-263):
json["device_id_origen"] = Util.DeviceName ?? "";
json["ip_origen"] = Util.getIpServerPermanente() ?? "";

// DESPUÉS:
json["device_id_origen"] = Util.DeviceName ?? "";
// Si hay IP de cliente REST (terminal cliente llamando al servidor), usar esa IP
como origen
// Si no hay (servidor directamente), usar IP del servidor
json["ip_origen"] = !string.IsNullOrEmpty(ipClienteRest)
    ? ipClienteRest
    : (Util.getIpServerPermanente() ?? "");
// Siempre enviar la IP del servidor para que PrinterServices SIEMPRE le notifique
json["ip_servidor"] = Util.getIpServerPermanente() ?? "";
```

IMPORTANTE: `ip_origen` ahora es la IP REAL del origen (cliente o servidor). `ip_servidor` es SIEMPRE la IP del servidor.

3.2.3 Mismo cambio para `EnviarDocumentoSingleAsync()`

Propagar `ipClienteRest` a los métodos de Fase 7B:

- `EnviarVentaAsync`
- `EnviarPromocionAsync`
- `EnviarEncuestaAsync`
- `EnviarPrecuentaAsync`

Todos delegan a `EnviarDocumentoSingleAsync` que a su vez llama a `EnriquecerImpresion`.

3.2.4 Agregar `IpClienteRest` al `ImpresionContext`

Archivo: `QuipuNetX/Services/Print/ImpresionContext.cs`

```
/// IP del cliente REST que originó la impresión (vacío si es el servidor
directamente)
public string IpClienteRest { get; set; }
```

Los factory methods (`CrearParaComandas` , `CrearParaVentaRapida` , etc.) deben aceptar este parámetro.

Fase 3: Modificar `PrinterServices` para priorizar `ip_origen` del JSON

3.3.1 Invertir prioridad en `PrintController`

Archivo: `PrinterServices/Api/Controllers/PrintController.cs`

Problema: Actualmente el `clientIp` del `RemoteEndPoint` (que siempre es `127.0.0.1` porque QuipuNet Servidor y `PrinterServices` corren en la misma PC) **sobrescribe** el `ip_origen` del JSON.

```
// ANTES (PostComanda, línea 36-39):
if (!string.IsNullOrEmpty(clientIp))
{
```

```

        job.IpOrigen = clientIp; // Siempre 127.0.0.1 → MALO
    }

    // DESPUÉS:
    // Priorizar ip_origen del JSON (viene del cliente real vía QuipuNet)
    // Solo usar clientIp del socket si el JSON no trae ip_origen
    if (string.IsNullOrEmpty(job.IpOrigen) && !string.IsNullOrEmpty(clientIp))
    {
        job.IpOrigen = clientIp; // Fallback al IP del socket
    }

```

Mismo cambio en:

- `PostComanda()` (línea ~36)
- `PostComandas()` (línea ~92)
- `PostVenta()` si existe lógica similar
- `PostPrecuenta()` si existe lógica similar

3.3.2 Parsear nuevo campo `ip_servidor` en `ParsePrintJob()`

```

// Agregar en ParsePrintJob():
job.IpServidor = GetString(json, "ip_servidor");

```

Fase 4: Doble notificación — Servidor + Cliente

3.4.1 Agregar campo `IpServidor` al modelo

Archivo: `PrinterServices/Queue/PrintJob.cs`

```

/// IP del QuipuNet Servidor (siempre se notifica el resultado aquí)
public string IpServidor { get; set; }

```

Archivo: `PrinterServices/Data/Models/PrintJobEntity.cs`

```

[Column("ip_servidor")]
public string IpServidor { get; set; } // IP del QuipuNet Servidor

```

3.4.2 Migración de BD

Archivo: `PrinterServices/Data/PrinterServiceDb.cs`

En `CreateTables()`, agregar migración para la nueva columna:

```

// Verificar columnas en print_jobs
var jobColumns = Query<dynamic>("PRAGMA table_info(print_jobs)");
bool hasIpServidor = false;
foreach (var col in jobColumns)
{
    var colDict = col as IDictionary<string, object>;
    if (colDict != null && colDict.ContainsKey("name"))
    {

```

```

        string colName = colDict["name"].ToString();
        if (colName == "ip_servidor") hasIpServidor = true;
    }
}
if (!hasIpServidor)
{
    Execute("ALTER TABLE print_jobs ADD COLUMN ip_servidor TEXT DEFAULT '');
    Log.Info("[DB] Columna 'ip_servidor' agregada a tabla print_jobs");
}

```

3.4.3 Modificar JobStatusCallbackNotifier

Archivo: PrinterServices/Notifications/JobStatusCallbackNotifier.cs

Cambiar NotifyStatusChangeAsync() para doble notificación:

```

public async Task NotifyStatusChangeAsync(PrintJob job, string newStatus, string
error = "")
{
    if (job == null) return;

    // 1. SIEMPRE notificar al servidor (ip_servidor)
    // Razón: El servidor mantiene PrintJobStatusManager global y necesita saber
    // el estado de TODOS los jobs, sin importar quién los originó.
    if (!string.IsNullOrEmpty(job.IpServidor))
    {
        var dto = BuildDto(job, newStatus, error);
        bool sent = await TrySendCallbackAsync(job.IpServidor, dto);
        if (!sent)
        {
            PersistFailedCallbackForIp(job, newStatus, error,
                "Servidor QuipuNetX offline o timeout", job.IpServidor);
        }
        else
        {
            Log.InfoFormat("[JOB-CALLBACK] ✅ Callback SERVIDOR enviado: job={0}
status={1} → {2}",
                job.JobId, newStatus, job.IpServidor);
        }
    }

    // 2. Notificar al cliente de origen SOLO si es diferente al servidor
    // Razón: El cliente que originó la comanda necesita saber si SU impresión
    // fue exitosa o falló, para mostrar badges/modales en su UI local.
    if (!string.IsNullOrEmpty(job.IpOrigen)
        && !IsLoopbackOrSameAsServer(job.IpOrigen, job.IpServidor))
    {
        var dto = BuildDto(job, newStatus, error);
        bool sent = await TrySendCallbackAsync(job.IpOrigen, dto);
        if (!sent)
        {
            PersistFailedCallbackForIp(job, newStatus, error,

```

```

        "Cliente QuipuNetX offline o timeout", job.IpOrigen);
    }
    else
    {
        Log.InfoFormat("[JOB-CALLBACK] ✅ Callback CLIENTE enviado: job={0} status={1} → {2}",
            job.JobId, newStatus, job.IpOrigen);
    }
}

/// Helper: verifica si es loopback o la misma IP del servidor
private bool IsLoopbackOrSameAsServer(string ipOrigen, string ipServidor)
{
    if (string.IsNullOrEmpty(ipOrigen)) return true;
    if (ipOrigen == "127.0.0.1" || ipOrigen == "::1" || ipOrigen == "localhost")
        return true;
    if (!string.IsNullOrEmpty(ipServidor) && ipOrigen == ipServidor) return true;
    return false;
}

```

3.4.4 Actualizar PersistFailedCallback para soportar IP destino

El método actual usa `job.IpOrigen` como destino. Necesitamos una variante que acepte un IP explícito:

```

private void PersistFailedCallbackForIp(PrintJob job, string newStatus, string
error,
    string sendError, string targetIp)
{
    // Mismo patrón que PersistFailedCallback pero usando targetIp en vez de
    job.IpOrigen
    // ...
    newCallback.IpOrigen = targetIp; // IP destino del callback (puede ser servidor
    o cliente)
    // ...
}

```

3.4.5 Actualizar JobStatusCallbackEntity

Considerar agregar un campo `tipo_destino` para distinguir callbacks al servidor vs al cliente:

```

[Column("tipo_destino")]
public string TipoDestino { get; set; } // "SERVIDOR" o "CLIENTE"

```

Fase 5: Columnas adicionales de trazabilidad en `print_jobs`

3.5.1 Nuevas columnas en `PrintJobEntity.cs`

```

[Column("ip_servidor")]
public string IpServidor { get; set; } // IP del QuipuNet Servidor (SIEMPRE se

```



```

notifica)

[Column("device_name_origen")]
public string DeviceNameOrigen { get; set; } // Nombre del dispositivo que originó
(ej: "TOSHIBA-PCJAROL")

[Column("tipo_documento")]
public string TipoDocumento { get; set; } // COMANDA, COMPROBANTE, PROMOCION,
ENCUESTA, PRECUENTA, DELIVERY

[Column("modalidad_venta")]
public string ModalidadVenta { get; set; } // salón, delivery, para llevar, etc.

```

3.5.2 Parsear en ParsePrintJob()

```

job.IpServidor = GetString(json, "ip_servidor");
job.DeviceNameOrigen = GetString(json, "device_name_origen");
job.TipoDocumento = GetString(json, "tipo_documento");
job.ModalidadVenta = GetString(json, "modalidad_venta");

```

3.5.3 Enviar desde EnriquecerImpresion() en QuipuNetX

```

json["ip_servidor"] = Util.getIpServerPermanente() ?? ""; // Siempre IP del
servidor
json["device_name_origen"] = Util.DeviceName ?? ""; // DeviceName del
servidor (o cliente si se propaga)
json["tipo_documento"] = impresion.Tipo ?? "comanda"; // Tipo de
documento
json["modalidad_venta"] = impresion.ModalidadVenta ?? ""; // Modalidad de
venta

```

3.5.4 Migraciones ALTER TABLE

Mismo patrón que Fase 4.2 para cada columna nueva.

Fase 6: Nuevos endpoints para consulta de estado por IP

3.6.1 Endpoints en PrinterServices

Archivo: PrinterServices/Api/ApiRouter.cs

Método	Endpoint	Descripción	Query
GET	/api/jobs/byip	Jobs de un IP específico	?ip=10.0.0.2&status=FAILED
GET	/api/jobs/bydevice	Jobs de un dispositivo	?device=TOSHIBA-PCJAROL
GET	/api/jobs/failed/byip	Jobs fallidos de un IP	?ip=10.0.0.2
GET	/api/jobs/stats/byip	Estadísticas por IP	?ip=10.0.0.2

Archivo: `PrinterServices/Api/Controllers/JobController.cs` (agregar métodos)

```
public ApiResult GetJobsByIp(string ip, string status = null)
{
    // SELECT * FROM print_jobs WHERE ip_origen = ? [AND estado = ?] ORDER BY
    fecha_creacion DESC LIMIT 50
}

public ApiResult GetFailedJobsByIp(string ip)
{
    // SELECT * FROM print_jobs WHERE ip_origen = ? AND estado IN
    ('FAILED','EXPIRED') ORDER BY fecha_creacion DESC
}

public ApiResult GetJobsByDevice(string deviceName)
{
    // SELECT * FROM print_jobs WHERE device_name_origen = ? ORDER BY fecha_creacion
    DESC LIMIT 50
}
```

3.6.2 Endpoints proxy en QuipuNet Servidor

Archivo: `QuipuNetX/ws/WebServer_New.cs` o nuevo módulo Nancy

Ruta: `/api/rest/printservices/...`

```
// Las terminales cliente llaman a estos endpoints del servidor
// El servidor hace proxy a PrinterServices y retorna el resultado

Get("/api/rest/printservices/jobs/failed", paramsGet =>
{
    // Obtener IP del cliente que hace la consulta
    var ipCliente = GetClientIp(this.Request) ?? "";
    // Proxy a PrinterServices: GET /api/jobs/failed/byip?ip={ipCliente}
    return ProxyToPrinterServices("/api/jobs/failed/byip?ip=" + ipCliente);
});

Get("/api/rest/printservices/jobs/status", paramsGet =>
{
    var ipCliente = GetClientIp(this.Request) ?? "";
    return ProxyToPrinterServices("/api/jobs/byip?ip=" + ipCliente);
});
```

Ventaja: El cliente NO necesita saber la IP de PrinterServices. Solo habla con QuipuNet Servidor.

Fase 7: Terminal Cliente recibe notificaciones

3.7.1 Prerequisito: Nancy escuchando en clientes

Las terminales cliente ya corren QuipuNet.exe con Nancy WebServer (puerto 8081). El endpoint `POST /api/rest/printerservice/updateJobStatus` en `PrinterServiceModule` ya existe y funciona tanto en servidor como en cliente.

3.7.2 Flujo en terminal cliente

```
PrinterServices (10.0.0.100)
  → HTTP POST http://10.0.0.2:8081/api/rest/printerservice/updateJobStatus
    → PC1 (Nancy) recibe callback
      → PrintJobStatusManager.UpdateStatus(jobId, "DONE")
        → Evento JobStatusChanged
          → UI: badge "Impresión OK" o "Impr. Fallida"
```

3.7.3 ¿Qué pasa si la terminal cliente NO tiene Nancy?

- **Tablets Android:** No tienen Nancy → PrinterServices NO podrá enviar callback HTTP
- **Solución:** El cliente puede **consultar** via REST al servidor (Fase 6 endpoints proxy)
- **Polling:** Timer cada 5s consultando `/api/rest/printservices/jobs/status`

3.7.4 Puerto configurable para clientes

Los clientes Windows sí tienen Nancy en :8081. PrinterServices usa este puerto hardcoded en `JobStatusCallbackNotifier` :

```
string port = _config.GetString("QuipuNetXPort", "8081");
```

Si algún cliente usa un puerto diferente, se puede agregar un campo `port_origen` al JSON de impresión.

Fase 8: Verificaciones del Router (no requiere cambios)

3.8.1 PedidoRouter en modo cliente

```
// PedidoRouter.enviarPedidos():
if (Util.esModoServidor())
{
    // Llama a PedidoController directamente → ipClienteRest = "" (es el servidor)
    PedidoController.enviarPedidos(...);
}
else
{
    // Llama via HTTP REST al servidor
    PedidoClient.enviarPedidos(...);
    // → El servidor intercepta la IP del socket TCP (10.0.0.2) vía GetClientIp()
    // → No necesita cambio en PedidoClient
}
```

No se necesita cambio en `PedidoClient` porque la IP del cliente se obtiene del lado del servidor (del socket HTTP).

4. Tabla de decisión: ¿Quién recibe notificación?

Escenario	ip_origen	ip_servidor	PS notifica a servidor	PS notifica a cliente
-----------	-----------	-------------	------------------------	-----------------------

Servidor directo (flag ON, sin REST)	10.0.0.100	10.0.0.100	✅ Sí	❌ No (es el mismo)
Cliente PC1 via REST	10.0.0.2	10.0.0.100	✅ Sí	✅ Sí (10.0.0.2)
Cliente PC2 via REST	10.0.0.3	10.0.0.100	✅ Sí	✅ Sí (10.0.0.3)
Tablet via REST	10.0.0.6	10.0.0.100	✅ Sí	✅ Intenta (puede fallar si no tiene Nancy)
Flag OFF	N/A	N/A	❌ No aplica	❌ No aplica

5. Resumen de archivos a modificar

QuipuNetX (Backend)

Archivo	Cambio	Fase
ws/WebServer_New.cs	GetClientIp() en endpoints de pedidos/ventas, pasar como parámetro	1
ws/modules/PedidoModule.cs	Igual para rutas Nancy	1
server/PedidoServer.cs	Agregar ipClienteRest como parámetro opcional	1
controller/PedidoController.cs	Propagar ipClienteRest hasta PrinterServiceClient	1
controller/VentaController.cs	Propagar ipClienteRest para ventas/delivery	1
Services/Print/PrinterServiceClient.cs	ipClienteRest en EnriquecerImpresion, EnviarComandasAsync, EnviarDocumentoSingleAsync; agregar ip_servidor al JSON	2
Services/Print/ImpresionContext.cs	Agregar propiedad IpClienteRest	2

PrinterServices

Archivo	Cambio	Fase
Api/Controllers/PrintController.cs	Invertir prioridad: JSON ip_origen > socket clientIp	3
Queue/PrintJob.cs	Nueva propiedad IpServidor	4
Data/Models/PrintJobEntity.cs	Nuevas columnas: ip_servidor, device_name_origen, tipo_documento, modalidad_venta	4, 5
Data/PrinterServiceDb.cs	ALTER TABLE migraciones	4, 5
Notifications/JobStatusCallbackNotifier.cs	Doble notificación: servidor + cliente	4
Api/ApiRouter.cs	Nuevas rutas /api/jobs/byip, /api/jobs/failed/byip, /api/jobs/bydevice	6

Api/Controllers/JobController.cs	Queries por IP/device	6
----------------------------------	-----------------------	---

QuipuNet Front (opcional, después)

Archivo	Cambio	Fase
ws/modules/PrinterServiceModule.cs	Verificar que funciona en modo cliente	7
Presenters de Pedidos/Ventas	Consultar status de impresiones vía endpoint proxy	7

6. Orden de implementación recomendado

MVP (Fases 1-4):

Fase 1: Propagar ipClienteRest (WebServer → Server → Controller)

↓

Fase 2: Propagar hasta PrinterServiceClient (ip_origen + ip_servidor en JSON)

↓

Fase 3: PrinterServices prioriza ip_origen del JSON sobre socket IP

↓

Fase 4: Doble notificación (servidor SIEMPRE + cliente si diferente)

Mejoras (Fases 5-8):

Fase 5: Columnas extras de trazabilidad (device_name, tipo_documento, modalidad)

↓

Fase 6: Endpoints de consulta por IP/device en PrinterServices + proxy en QuipuNet

↓

Fase 7: Verificar que terminales cliente reciben callbacks

↓

Fase 8: Verificar que PedidoRouter/Client no necesita cambios

Las Fases 1-4 son el MVP: con eso, PrinterServices ya sabe quién originó cada impresión y notifica tanto al servidor como al cliente real.

7. Diagrama de flujo completo (post-implementación)

```

t=0ms    PC1 (10.0.0.2) → REST POST /api/rest/pedido/enviarPedidos → QuipuNet Servidor (10.0.0.100)
          |
t=5ms    WebServer_New.cs: ipClienteRest = GetClientIp(request) = "10.0.0.2"
          |
t=10ms   PedidoServer.enviarPedidos(data, esMovil, compress, "10.0.0.2")
          |
t=15ms   PedidoController.addLista(..., ipClienteRest: "10.0.0.2")
          |
t=20ms   PrinterServiceClient.EnviaComandasAsync(..., ipClienteRest: "10.0.0.2")
          |
t=25ms   EnriquecerImpresion():
  
```

```

|   json["ip_origen"]   = "10.0.0.2"      ← IP del cliente (PC1)
|   json["ip_servidor"] = "10.0.0.100"    ← IP del servidor (siempre)
|
t=30ms HTTP POST → PrinterServices (localhost:8090) /api/print/comandas
|
t=35ms PrintController.PostComandas():
|   job.IpOrigen = "10.0.0.2"             ← Del JSON (prioridad)
|   job.IpServidor = "10.0.0.100"        ← Del JSON
|   _jobManager.Enqueue(job)
|
t=40ms Response → QuipuNet → PC1: { status: "OK", jobs: [{job_id: "714"}] }
|
t=500ms PrintWorker → TCP:9100 → Impresora (10.0.0.12)
|
t=800ms Impresión exitosa → MarkDone(job)
|
t=805ms JobStatusCallbackNotifier.NotifyStatusChangeAsync(job, "DONE"):
|   1. POST http://10.0.0.100:8081/api/rest/printerservice/updateJobStatus ←
SERVIDOR ✓
|   2. POST http://10.0.0.2:8081/api/rest/printerservice/updateJobStatus ←
CLIENTE PC1 ✓
|
t=810ms QuipuNet Servidor (10.0.0.100): PrintJobStatusManager.UpdateStatus("714",
"DONE")
|   → Evento AllJobsDone → UI del servidor actualizada
|
t=812ms PC1 Cliente (10.0.0.2): PrintJobStatusManager.UpdateStatus("714", "DONE")
|   → Evento AllJobsDone → UI del cliente actualizada
|   → Badge "Impr. Fallida" desaparece si existía

```

8. Edge cases

8.1 Servidor imprime directamente (sin REST de cliente)

```

ip_origen = "10.0.0.100" (IP del servidor)
ip_servidor = "10.0.0.100" (IP del servidor)
→ IsLoopbackOrSameAsServer() retorna true
→ Solo 1 notificación (al servidor)

```

8.2 Múltiples clientes imprimen simultáneamente

```

PC1 (10.0.0.2) envía comanda → job_id 714, ip_origen=10.0.0.2
PC2 (10.0.0.3) envía comanda → job_id 715, ip_origen=10.0.0.3

Job 714 DONE → notifica a 10.0.0.100 + 10.0.0.2
Job 715 FAILED → notifica a 10.0.0.100 + 10.0.0.3

PC1 solo ve estado de SUS jobs → ✓
PC2 solo ve estado de SUS jobs → ✓
Servidor ve TODOS los jobs → ✓

```

8.3 Cliente se desconecta después de enviar

PC1 (10.0.0.2) envía comanda → apaga PC
Job 714 DONE → notifica a 10.0.0.100 ✅ + intenta 10.0.0.2 ❌ timeout
→ Callback para 10.0.0.2 se persiste en BD para retry
→ Si PC1 nunca vuelve → después de 10 reintentos → FALLIDO (pero el servidor ya sabe)
→ Cuando PC1 vuelve → Initialize() sincroniza estados desde PrinterServices

8.4 Tablet Android sin Nancy

Tablet (10.0.0.6) envía pedido → ip_origen=10.0.0.6
Job 714 DONE → notifica a 10.0.0.100 ✅ + intenta 10.0.0.6:8081 ❌ connection refused
→ Callback se persiste para retry (fallará siempre)
→ Tablet puede consultar estado via REST al servidor (Fase 6 endpoints proxy)

9. Restricciones y reglas

1. **ip_origen** = IP del origen REAL de la impresión (cliente o servidor)
2. **ip_servidor** = IP del QuipuNet Servidor (SIEMPRE se notifica aquí)
3. **PrinterServices SIEMPRE notifica al servidor** — es la fuente de verdad global
4. **PrinterServices notifica al cliente SOLO si es diferente al servidor** y no es loopback
5. **No se modifica PedidoClient** — la IP del cliente se obtiene del lado del servidor (socket HTTP)
6. **Backward compatible:** Si **ip_servidor** viene vacío (versión vieja), PrinterServices funciona como antes (solo **ip_origen**)
7. **JSON tiene prioridad:** **ip_origen** del JSON > **clientIp** del socket (socket siempre será 127.0.0.1)
8. **Parámetro opcional:** **ipClienteRest** = "" en toda la cadena, para no romper callers existentes
9. **FeatureFlag USAR_PRINTER_SERVICE: Solo el FRONT lo valida** — El Backend (Controllers, Servers, PrinterServiceClient) **NO verifica** si la terminal es servidor o cliente. El parámetro **ipClienteRest** viaja como dato transparente por toda la cadena sin importar el rol. El **FeatureFlag USAR_PRINTER_SERVICE** solo lo evalúa el **Front** (Presenters) para decidir si mostrar modales de PrinterServices o imprimir localmente. Tanto terminales servidor como cliente activan el flag en el Front si está configurado — la diferencia es que el servidor llama al Controller directo y el cliente llama via REST, pero ambos terminan en el mismo Controller con el mismo **ipClienteRest** .
10. **Código nuevo NO rompe lo existente** — Todo parámetro nuevo es **opcional con default vacío** (`string ipClienteRest = ""`). Si un caller existente no pasa el parámetro, el flujo funciona exactamente igual que antes (**ip_origen** = IP del servidor). Solo cuando el endpoint HTTP pasa un **ipClienteRest** diferente de vacío, el comportamiento cambia.

10. Métricas de éxito

- ☐ Cuando un cliente envía pedido, **print_jobs.ip_origen** tiene la IP del cliente (no del servidor)
- ☐ Cuando el servidor envía directo, **print_jobs.ip_origen** tiene la IP del servidor
- ☐ **print_jobs.ip_servidor** SIEMPRE tiene la IP del servidor
- ☐ Callback DONE/FAILED llega al servidor en todos los casos
- ☐ Callback DONE/FAILED llega al cliente cuando el cliente tiene Nancy activo

- ☐ Dashboard de PrinterServices muestra IP real de origen por cada job
- ☐ Terminales cliente pueden consultar estado de SUS impresiones vía REST proxy

11. Feature Flag Detection desde Terminal Cliente (IMPLEMENTADO)

Estado:  IMPLEMENTADO — Build verificado 0 errores
Fecha: 2026-03-08

11.1 Problema

Cuando una terminal opera en **modo cliente**, necesita saber si el **servidor** tiene el feature flag `USAR_PRINTER_SERVICE` activo. Esto determina si el cliente debe mostrar UI de estado de impresión (badges, modales) o asumir impresión local.

El flag `USAR_PRINTER_SERVICE` vive en el archivo `config_fla.cfg` del **servidor**, no del cliente. El cliente no puede leerlo directamente.

11.2 Solución

Al iniciar la app en modo cliente (tanto login fresco como reapertura con sesión), se consulta al servidor vía REST y se guarda el resultado en una variable global del Front.

11.3 Punto de convergencia

Ambos flujos (login fresco + reapertura con sesión) convergen en `MenuPrincipalview` :

```
CASO 1: Login fresco → LoginPrincipalNew.ShowSuccesLogin() → irAMainForm() → new
MenuPrincipalview()
CASO 2: Reapertura → MainWindowsNew.estaLogueado() = true → new MenuPrincipalview()
                                     |
                                     Window_Loaded (async)
                                     |
                                     await
checkearSiServidorTienePrinterServicesActivo()
```

11.4 Archivos modificados

Archivo	Cambio
QuipuNet/Utils/UtilFront.cs	Nueva propiedad estática <code>FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES</code> (bool, default false)
QuipuNet/Xaml/MenuPrincipalview.xaml.cs	Nuevo método async <code>checkearSiServidorTienePrinterServicesActivo()</code> + invocación en <code>Window_Loaded</code>

11.5 Variable global

```
// En UtilFront.cs
QuipuNet.Utils.UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES // bool, default
```



```
false
```

11.6 Comportamiento del método

1. **Solo modo cliente:** Si `Util.esModoCliente()` es `false`, retorna inmediatamente.
2. **Una sola vez:** Campo estático `_printerServiceFlagChecked` impide re-ejecución.
3. **No bloquea UI:** `Task.Run()` ejecuta la petición HTTP en hilo de fondo.
4. **Timeout seguro:** Si el servidor no responde, deja `false` (impresión local).
5. **Endpoint:** GET `http://{ipServidor}:8081/api/rest/featureflags/listar`
6. **Parseo:** Busca en el array `flags` el objeto con `nombre == "USAR_PRINTER_SERVICE"` y lee `habilitado`.
7. **Usa WebClient:** `System.Net.WebClient` (disponible en el Front, no RestSharp).

11.7 Cómo usar en Presenters

```
// En cualquier Presenter del Front (modo cliente):
if (Util.esModoCliente() && UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES)
{
    // El servidor tiene PrinterServices activo
    // → Mostrar UI de estado de impresión (badges, modales, etc.)
    // → NO imprimir localmente, el servidor ya delegó a PrinterServices
}
else
{
    // Comportamiento normal: imprimir localmente si hay ImpresionList
}
```

11.8 Edge cases

Escenario	Resultado
Servidor apagado o sin red	false (impresión local, comportamiento conservador)
Servidor sin flag definido	false (flag no encontrado en la lista)
Servidor con flag false	false (se respeta la configuración del servidor)
Servidor con flag true	true (cliente activa UI de PrinterServices)
Modo servidor (no cliente)	No se ejecuta el check, variable queda en false
Segunda apertura de MenuPrincipalview	No re-ejecuta (control <code>_printerServiceFlagChecked</code>)

12. Fase 9: PrintJobCallbackServer — Mini Nancy para Clientes

Estado:  IMPLEMENTADO — Build verificado 0 errores (QuipuNetX x86, QuipuNet Front, PrinterServices)

Fecha: 2026-03-08

Objetivo: Que terminales cliente reciban callbacks instantáneos de PrinterServices sin depender del WebServer_New completo, sin BD, y con históricos vía REST al servidor.

12.1 Problema

El `WebServer_New` (Nancy completo en `:8081`) **solo se inicializa en modo servidor** (`SyncService.verificarEstadoServidorHttp()` → `if (Util.esModoServidor())`). Las terminales cliente **NO tienen Nancy corriendo**, por lo tanto **NO pueden recibir callbacks HTTP** de `PrinterServices`.

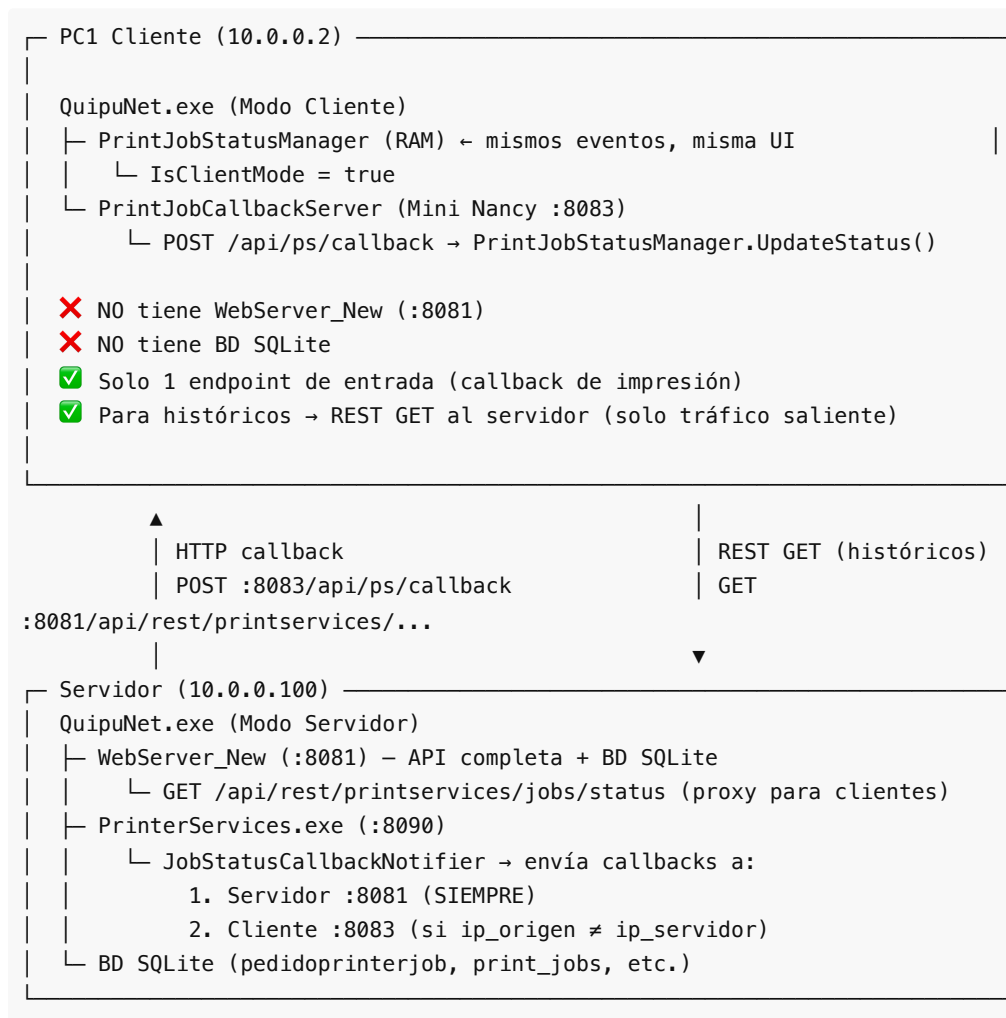
Sin callbacks, el cliente no se entera en tiempo real de si su impresión fue `DONE/FAILED/EXPIRED`.

12.2 Solución: Mini Nancy Server Aislado

Crear un **servidor Nancy ultraliviano** dedicado exclusivamente a recibir callbacks de `PrinterServices` en terminales cliente. Características:

- **Puerto fijo hardcodeado: 8083** — Todos los clientes usan el mismo puerto
- **UN solo endpoint:** `POST /api/ps/callback`
- **NO accede a BD** — Solo actualiza `PrintJobStatusManager` (RAM)
- **NO tiene endpoints de negocio** — Es exclusivo para callbacks de impresión
- **Aislado del WebServer_New** — No interfiere con nada existente

12.3 Arquitectura



12.4 Puerto Hardcodeado: 8083

Decisión: Todos los clientes usan el puerto **8083**. No es configurable. Razones:

- Simplifica la lógica: PrinterServices sabe que todo cliente escucha en **:8083**
- No necesita campo `port_origen` en el JSON de impresión
- El servidor escucha en **:8081** (WebServer_New) — no hay conflicto
- El **:8083** es solo para callbacks, no tiene endpoints de negocio

Constante: `PrintJobCallbackServer.CALLBACK_PORT = 8083`

12.5 Archivos a crear/modificar

Archivo	Acción	Descripción
QuipuNetX/Services/Print/PrintJobCallbackServer.cs	CREAR	Mini Nancy server + módulo callback
QuipuNetX/Services/Print/PrintJobStatusManager.cs	MODIFICAR	Agregar <code>InitClientMode()</code> , <code>StopClientMode()</code> , <code>IsClientMode</code>
QuipuNetX/Client/PrintJobStatusClient.cs	CREAR	Client REST para consultar históricos al servidor
QuipuNetX/ws/modules/PrinterServiceModule.cs	MODIFICAR	Guard <code>Util.esModoServidor()</code> antes de acceder BD
QuipuNet/Xaml/MenuPrincipalview.xaml.cs	MODIFICAR	Llamar <code>InitClientMode()</code> después de detectar feature flag

12.6 PrintJobCallbackServer.cs — Diseño completo

```
// QupuNetX/Services/Print/PrintJobCallbackServer.cs
namespace QupuNetX.Services.Print
{
    /// <summary>
    /// Mini Nancy server que SOLO recibe callbacks de PrinterServices.
    /// Se usa exclusivamente en modo CLIENTE.
    ///
    /// - Puerto fijo: 8083 (hardcodeado)
    /// - UN solo endpoint: POST /api/ps/callback
    /// - NO accede a BD
    /// - Solo actualiza PrintJobStatusManager (RAM)
    /// </summary>
    public class PrintJobCallbackServer : IDisposable
    {
        public const int CALLBACK_PORT = 8083;    // Puerto fijo para todos los
        clientes

        private NancyHost _host;
        private bool _running;

        public bool IsRunning => _running;

        public void Start() { ... }    // Inicia en :8083
    }
}
```

```

        public void Stop() { ... }    // Detiene
        public void Dispose() { ... } // Cleanup
    }

    /// <summary>
    /// Módulo Nancy ultraliviano: UN solo endpoint para callbacks.
    /// NO hereda de CorsModuleBase, NO accede a BD.
    /// </summary>
    public class PrintJobCallbackModule : NancyModule
    {
        public PrintJobCallbackModule() : base("/api/ps")
        {
            // ÚNICO endpoint
            Post("/callback", _ => ProcessCallback());
        }

        // Parsea JSON → PrintJobStatusManager.Instance.UpdateStatus()
        // JAMÁS accede a BD
    }
}

```

12.7 PrintJobStatusManager — Nuevos miembros para modo cliente

```

// NUEVOS miembros en PrintJobStatusManager.cs

private PrintJobCallbackServer _callbackServer;    // Mini server para modo cliente
private bool _isClientMode = false;               // ¿Estamos en modo cliente?

/// Inicializa modo cliente: arranca mini Nancy en :8083
public void InitClientMode()
{
    _isClientMode = true;
    _callbackServer = new PrintJobCallbackServer();
    _callbackServer.Start();
}

/// Detiene modo cliente y libera recursos
public void StopClientMode()
{
    _isClientMode = false;
    _callbackServer?.Stop();
    _callbackServer?.Dispose();
    _callbackServer = null;
}

/// Indica si estamos en modo cliente
public bool IsClientMode => _isClientMode;

```

12.8 PrintJobStatusClient.cs — Consulta de históricos al servidor

```
// QuipuNetX/Client/PrintJobStatusClient.cs
namespace QuipuNetX.Client
{
    /// <summary>
    /// Client REST para que terminales en modo cliente consulten
    /// históricos de impresión al servidor. Solo tráfico saliente.
    /// La BD está en el servidor, el cliente JAMÁS accede a BD.
    /// </summary>
    public static class PrintJobStatusClient
    {
        /// Consulta estado de jobs específicos al servidor
        public static void getJobStatuses(string jobIdsCsv, Action<Respuesta>
callback) { ... }

        /// Consulta jobs fallidos del cliente al servidor
        public static void getFailedJobs(Action<Respuesta> callback) { ... }

        /// Consulta histórico por pedidoId al servidor
        public static void getJobByPedidoId(string pedidoId, Action<Respuesta>
callback) { ... }
    }
}
```

12.9 Endpoint proxy en servidor para históricos

```
// En WebServer_New.cs o PrinterServiceModule.cs del SERVIDOR:

// GET /api/rest/printservices/jobs/status?job_ids=abc,def
// → Consulta PrintJobStatusManager (RAM) del servidor y responde

// GET /api/rest/printservices/jobs/history?pedido_id=1772
// → Consulta tabla pedidoprinterjob (BD) del servidor y responde
```

12.10 Guard en PrinterServiceModule para modo cliente

```
// En PrinterServiceModule.cs, método ProcessUpdateJobStatus():
// ANTES:
ActualizarEstadoEnBD(jobId, status);

// DESPUÉS:
// Solo persistir en BD si estamos en modo servidor (que tiene BD)
// En modo cliente, el PrintJobCallbackModule maneja los callbacks
// y solo actualiza RAM vía PrintJobStatusManager.UpdateStatus()
if (Util.esModoServidor())
{
    ActualizarEstadoEnBD(jobId, status);
}
```

12.11 Integración en ciclo de vida del cliente

```
// En MenuPrincipalview.xaml.cs, método Window_Loaded:
// Después de checkearSiServidorTienePrinterServicesActivo():

if (Util.esModoCliente() && UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES)
{
    // El servidor tiene PrinterServices activo → arrancar mini server para
    // callbacks
    PrintJobStatusManager.Instance.InitClientMode();
}
```

```
// Al cerrar sesión o salir de la app:
PrintJobStatusManager.Instance.StopClientMode();
```

12.12 JobStatusCallbackNotifier — Puerto 8083 para clientes

En `PrinterServices/Notifications/JobStatusCallbackNotifier.cs`, al construir la URL de callback:

```
// Para el SERVIDOR: siempre puerto 8081 (WebServer_New)
string serverPort = _config.GetString("QuipuNetXPort", "8081");
string serverUrl = $"http://{job.IpServidor}:
{serverPort}/api/rest/printerservice/updateJobStatus";

// Para el CLIENTE: siempre puerto 8083 (PrintJobCallbackServer)
string clientPort = "8083"; // Hardcodeado, todos los clientes usan 8083
string clientUrl = $"http://{job.IpOrigen}:{clientPort}/api/ps/callback";
```

12.13 Flujo completo (post-implementación)

```
t=0ms    PC1 (10.0.0.2, modo cliente) → REST POST → Servidor (10.0.0.100:8081)
        | PedidoClient.enviarPedidos() → PedidoServer → PedidoController
        |
t=20ms    Servidor: PrinterServiceClient.EnviaComandasAsync()
        | json["ip_origen"] = "10.0.0.2" ← IP del cliente
        | json["ip_servidor"] = "10.0.0.100" ← IP del servidor
        |
t=30ms    HTTP POST → PrinterServices (localhost:8090)
        | job.IpOrigen = "10.0.0.2"
        | job.IpServidor = "10.0.0.100"
        |
t=40ms    Response → Servidor → PC1: { jobs: [{job_id: "714", pedido_ids: ["1772"]}]}
        |
t=45ms    PC1: PrintJobStatusManager.Instance.RegisterJobs(...)
        | → Status = PENDING en RAM del cliente
        |
t=500ms    PrintWorker → TCP:9100 → Impresora
        |
t=800ms    Impresión exitosa → MarkDone(job)
```

t=805ms	JobStatusCallbackNotifier: 1. POST http://10.0.0.100:8081/api/rest/printerservice/updateJobStatus ← 2. POST http://10.0.0.2:8083/api/ps/callback ←
SERVIDOR	
CLIENTE	
t=810ms	Servidor: PrintJobStatusManager.UpdateStatus("714", "DONE") + ActualizarEstadoEnBD("714", "DONE") ← Persiste en BD
t=812ms	PC1 Cliente: PrintJobCallbackModule recibe callback → PrintJobStatusManager.Instance.UpdateStatus("714", "DONE") → NO accede a BD (jamás) → Evento AllJobsDone → UI del cliente se actualiza instantáneamente

12.14 Tabla comparativa: Servidor vs Cliente

Aspecto	Modo Servidor	Modo Cliente
WebServer_New (:8081)	✅ Activo	❌ No existe
PrintJobCallbackServer (:8083)	❌ No necesita	✅ Activo
BD SQLite	✅ Accede	❌ Jamás
Callbacks de PS	Vía :8081 updateJobStatus	Vía :8083 /api/ps/callback
PrintJobStatusManager	RAM + BD	Solo RAM
Históricos	Consulta local (BD)	REST GET al servidor
Eventos UI	JobStatusChanged, etc.	Idénticos
UI (badges, modales)	Completa	Idéntica

12.15 Edge cases del mini server

Escenario	Comportamiento
Puerto 8083 ocupado	Log error, cliente funciona sin callbacks (puede usar polling como fallback)
Firewall bloquea 8083	PrinterServices no puede enviar callback → persiste para retry
Cliente se apaga	Callbacks fallan → servidor ya tiene el estado (fuente de verdad)
Múltiples clientes simultáneos	Cada uno en su :8083, IPs diferentes → sin conflicto
Cliente consulta histórico	REST GET al servidor → servidor responde con datos de BD
Servidor se apaga	Cliente pierde históricos, pero RAM local conserva estado actual

12.16 Restricciones adicionales

11. **Puerto 8083 hardcodedo** — Todos los clientes escuchan en `:8083` para callbacks. No es configurable.
12. **PrintJobCallbackServer es independiente** — No comparte NancyHost, módulos, ni configuración con WebServer_New.
13. **PrintJobCallbackModule NO hereda de CorsModuleBase** — Es un `NancyModule` puro sin dependencias del servidor.
14. **Modo cliente JAMÁS accede a BD** — Ni SQLite local ni remota directamente. Para datos históricos, consulta al servidor vía REST.
15. **InitClientMode() se llama UNA VEZ** — Al detectar que el servidor tiene PrinterServices activo, en MenuPrincipalview.
16. **StopClientMode() al cerrar sesión** — Libera el puerto 8083 y detiene el mini server.

12.17 Implementación real — Archivos creados y modificados

Archivos CREADOS

Archivo	Proyecto	Descripción
QuipuNetX/Services/Print/PrintJobCallbackServer.cs	QuipuNetX	Mini Nancy server (:8083) + CallbackOnlyBootstrapper + PrintJobCallbackModule
QuipuNetX/Client/PrintJobStatusClient.cs	QuipuNetX	Client REST para históricos (getJobStatuses, getFailedJobs, getJobByPedidoId)

Archivos MODIFICADOS

Archivo	Proyecto	Cambio
QuipuNetX/Services/Print/PrintJobStatusManager.cs	QuipuNetX	Nuevos: <code>_callbackServe</code> , <code>_isClientMode</code> , <code>InitClientMode()</code> , <code>StopClientMode()</code> , <code>IsClientMode</code>
QuipuNetX/ws/modules/PrinterServiceModule.cs	QuipuNetX	Guard <code>Util.esModoServidor()</code> antes de <code>ActualizarEstadoEnBD</code> + endpoints proxy GET (jobs/status, jobs/fail, jobs/bypedido)
QuipuNetX/QuipuNetX.csproj	QuipuNetX	Registrados <code>PrintJobCallbackServer</code> y <code>PrintJobStatusClient</code> en <code><Compile></code>
QuipuNet/Xaml/MenuPrincipalview.xaml.cs	QuipuNet (Front)	<code>InitClientMode()</code> después de detectar feature flag activo + <code>StopClientMode</code> en <code>Window_Closed</code>

PrinterServices/Notifications/JobStatusCallbackNotifier.cs	PrinterServices	TrySendCallbackAsync ahora distingue servicio (:8081) vs cliente (:8083) con parámetro isClient
--	-----------------	--

Cambio en PrinterServices — JobStatusCallbackNotifier

Problema: TrySendCallbackAsync() usaba siempre puerto 8081 y endpoint /api/rest/printerservice/updateJobStatus para TODOS los destinos. Pero el cliente no tiene WebServer_New en :8081, tiene el mini Nancy en :8083.

Solución: Nuevo parámetro bool isClient = false en TrySendCallbackAsync() :

```
// SERVIDOR (isClient=false):
http://{ip}:8081/api/rest/printerservice/updateJobStatus
// CLIENTE (isClient=true): http://{ip}:8083/api/ps/callback
```

Llamadas actualizadas en NotifyStatusChangeAsync() :

1. **Callback al servidor:** TrySendCallbackAsync(job.IpServidor, dto, false) → :8081
2. **Callback al cliente:** TrySendCallbackAsync(job.IpOrigen, dto, true) → :8083
3. **Fallback (backward compat):** TrySendCallbackAsync(job.IpOrigen, dto, false) → :8081

Hallazgo crítico: Nancy auto-descubrimiento de módulos

Problema descubierto durante implementación: Nancy auto-descubre por reflexión TODOS los NancyModule del assembly. Sin protección, el mini server del cliente expondría PedidoModule, ClienteModule, OrdenpedidoModule, etc. — endpoints de negocio accesibles desde la red.

Solución implementada: CallbackOnlyBootstrapper con BeforeRequest pipeline que retorna 404 a cualquier ruta que no sea POST /api/ps/callback . Nancy carga los módulos internamente, pero ninguna petición llega a ellos.

```
// En CallbackOnlyBootstrapper.ApplicationStartup():
pipelines.BeforeRequest.AddItemToStartOfPipeline(ctx =>
{
    if (method == "POST" && path == "/api/ps/callback")
        return null; // Permitir
    return HttpStatusCode.NotFound; // Bloquear todo lo demás
});
```

Se intentó primero NancyInternalConfiguration.WithOverrides(x => x.ModuleCatalog = ...) pero Nancy 2.0 no tiene ModuleCatalog en NancyInternalConfiguration → error CS1061 . El approach de BeforeRequest pipeline es más robusto y compatible.

12.18 Endpoints proxy del servidor para clientes

Agregados en PrinterServiceModule.cs del servidor:

Método	Ruta completa	Fuente de datos	Descripción
--------	---------------	-----------------	-------------

GET	/api/rest/printerservice/jobs/status?job_ids=714,715	RAM (PrintJobStatusManager)	Estado actual de jobs específicos
GET	/api/rest/printerservice/jobs/failed	RAM (PrintJobStatusManager)	Todos los jobs FAILED/EXPIRED
GET	/api/rest/printerservice/jobs/bypedido?pedido_id=1772	BD (PedidoprinterjobController)	Histórico por pedidoid

Uso desde el cliente: `PrintJobStatusClient.getJobStatuses()` , `getFailedJobs()` , `getJobByPedidoId()` — solo tráfico saliente, el cliente JAMÁS accede a BD.

12.19 Build verificado

Proyecto	Plataforma	Resultado
QuipuNetX	x86 Debug	✅ 0 errores
QuipuNetX	AnyCPU Debug	✅ 0 errores
QuipuNet (Front)	Debug	✅ 0 errores
PrinterServices	Debug	✅ 0 errores

13. Fase 10: Activación de UI de PrinterServices en Modo Cliente (IMPLEMENTADO)

Estado: ✅ IMPLEMENTADO — Build verificado 0 errores

Fecha: 2026-03-09

Objetivo: Que las terminales en modo cliente activen la misma UI de PrinterServices que el servidor (modales, badges, animaciones, suscripción a eventos) cuando el servidor tiene

`USAR_PRINTER_SERVICE` activo.

13.1 Problema

Los Presenters del Front verificaban

`FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE")` para bifurcar entre flujo PS y flujo local. Pero `FeatureFlagManager` lee el archivo `config_flg.cfg` **local** — en una terminal cliente, ese archivo tiene `USAR_PRINTER_SERVICE=false` (o no existe). El cliente nunca activaba la UI de PrinterServices aunque el servidor Sí tuviera el flag activo.

13.2 Solución

Agregar en cada punto de bifurcación la condición:

```
|| (Util.esModoCliente() && UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES)
```

Donde `UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES` fue seteado al iniciar la app (Fase 11, sección 11) consultando al servidor vía REST `GET /api/rest/featureflags/listar`.

13.3 Prerequisito: Variable global y detección (sección 11)

Componente	Archivo	Descripción
Variable global	QuipuNet/Utils/UtilFront.cs	FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES (bool, default false)
Detección	QuipuNet/Xaml/MenuPrincipalview.xaml.cs	checkearSiServidorTienePrinterServicesActiv en Window_Loaded
Mini server	PrintJobStatusManager.InitClientMode()	Arranca :8083 si flag activo

13.4 Archivos modificados — 7 archivos, 12 puntos

13.4.1 ListaPedidosTemporalesPresenter.cs — 3 puntos

Método	Línea aprox	Contexto
enviarPedidos() callback	~137	ImpresionList vacía → verificarPrintJobsPorComandas
enviarPedidosOld() callback	~319	Mismo patrón
enviarPedidosSelfService() callback	~353	Mismo patrón

```
// ANTES:  
else if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE"))  
  
// DESPUÉS:  
// Fase 9: También aplica en modo cliente si el servidor tiene PS activo  
else if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE")  
    || (Util.esModoCliente() &&  
        UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES))
```

13.4.2 FragmentDetallePedidoPresenter.cs — 1 punto + using

- Agregado using QuipuNet.Utils; (no lo tenía)
- Reimpresión de pedidos → verificarPrintJobsPorComandas()

13.4.3 ListaDeliveryPendientePresenter.cs — 1 punto

- Confirmar delivery pendiente → verificarPrintJobsPorComandas()

13.4.4 ResumenDeliveryPresenter.cs — 2 puntos

Método	Contexto
enviarPedidos() callback	Comandas delivery inmediato → verificarPrintJobsPorComandas()
cobrarDelivery() callback	Venta delivery → verificarPrintJobsPorVenta()

13.4.5 DeliverysConfirmadosNewPresenter.cs — 2 puntos

Método	Contexto
cobrarDelivery() callback	Venta delivery confirmado → verificarPrintJobsPorVenta()
enviarPedidos() callback	Comandas delivery → verificarPrintJobsPorComandas()

13.4.6 UtilesCabecera_New.xaml.cs — 1 punto

- ConfigureImpresiones() : Hace visible el componente de gestión de impresiones
- Usa QuipuNet.Utills.UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES (fully qualified)

13.4.7 UtilesCabeceraViewModel.cs — 3 puntos

Método	Contexto
ConfigurarVisibilidadGestionImpresiones()	Mostrar/ocultar botón de gestión de impresiones en cabecera
SubscribeToQueueEvents()	Suscribir a PrintJobStatusManager (modo PS) vs FailedQueueManager (modo legacy)
OcultarProgresoImpresion()	Verificar fallos en PrintJobStatusManager vs FailedQueueManager

13.5 Por qué el Backend (QuipuNetX Controllers) NO necesita cambios

Los Controllers (PedidoController , VentaController , DeliveryController) usan:

```
if (Util.esModoServidor() &&
    SQLite.FeatureFlagConfigReader.IsEnabled("USAR_PRINTER_SERVICE"))
```

Esto **NO necesita cambio** porque:

1. El Controller **siempre corre en el servidor** (el cliente llega vía REST)
2. Cuando un cliente envía un pedido → REST → el servidor ejecuta el Controller
3. El servidor evalúa **su propio** flag local → decide si delegar a PrinterServices
4. La respuesta vuelve al cliente con ImpresionList vacía (si PS activo) o con datos (si no)
5. El **Front del cliente** es quien necesita saber el flag para mostrar la UI correcta

13.6 PrintJobStatusManager es modo-agnóstico

PrintJobStatusManager funciona **idéntico** en ambos modos:

Aspecto	Servidor	Cliente
RegisterJobs()	RAM	RAM
UpdateStatus()	RAM + eventos	RAM + eventos
Eventos (JobStatusChanged, etc.)	Disparan UI	Disparan UI
Quién recibe callbacks	PrinterServiceModule :8081	PrintJobCallbackServer :8083
Ambos llaman a	UpdateStatus()	UpdateStatus()

BD	PrinterServiceModule persiste	Mini Nancy jamás toca BD
----	-------------------------------	---------------------------------

No se necesitó cambio en `PrintJobStatusManager` para soportar modo cliente — su diseño RAM-only lo hace inherentemente compatible.

13.7 Build verificado

Proyecto	Resultado
QuipuNet (Front)	✅ 0 errores CS

14. Fix: IpServidor vacío + Badge IsPrinterComandaFailed en modo cliente

Estado: ✅ IMPLEMENTADO — Build verificado 0 errores

Fecha: 2026-03-10

14.1 Bug: IpServidor llegaba vacío a PrinterServices

`Util.getServerPermanente()` retornaba vacío en ciertos momentos → `json["ip_servidor"]` viajaba como `""` → `PrinterServices` guardaba `IpServidor=""` en el job → al notificar, el paso 1 (servidor) se saltaba porque `string.IsNullOrEmpty("")` = true → la BD `pedidoprinterjob` del servidor NUNCA se actualizaba con EXPIRED/FAILED.

14.2 Fix: Fallback a 127.0.0.1

En `JobStatusCallbackNotifier.NotifyStatusChangeAsync()` :

```
string ipServidor = !string.IsNullOrEmpty(job.IpServidor) ? job.IpServidor :
"127.0.0.1";
```

`PrinterServices` SIEMPRE corre en la misma máquina que `QuipuNet Servidor` → `localhost` es correcto como fallback.

14.3 Orden de notificación (obligatorio)

1. **Primero** → `QuipuNet Servidor` (:8081 en `ipServidor` o `127.0.0.1`) → actualiza BD `pedidoprinterjob`
2. **Después** → `QuipuNet Cliente` (:8083 en `IpOrigen`) → actualiza RAM + eventos UI

14.4 Badge IsPrinterComandaFailed en modo cliente

Al entrar a una mesa, `ListaPedidos.MarcasPedidosConFalloDeImpresion()` consulta al servidor vía REST:

```
GET /api/rest/printerservice/jobs/failed-pedidos?pedido_ids=50992,50993,50994
→ Servidor consulta BD pedidoprinterjob → retorna ["50992"] (los que tienen
FAILED/EXPIRED)
→ Cliente marca p.IsPrinterComandaFailed = true → badge rojo visible
```

14.5 Archivos modificados

Archivo	Proyecto	Cambio
Notifications/JobStatusCallbackNotifier.cs	PrinterServices	Fallback IpServidor → 127.0.0.1 + usar variable ipServidor en todo el paso 1
ws/modules/PrinterServiceModule.cs	QuipuNetX	Endpoint GET /jobs/failed-pedidos consulta BD batch
Client/PrintJobStatusClient.cs	QuipuNetX	Método getFailedPedidoIds() consulta REST al servidor
MainApp/venta/Pedidos/ListaPedidos.cs	QuipuNet	MarcarPedidosConFalloDeImpresion modo cliente usa REST + OnJobStatusChanged guard BD

14.6 Fix: Modal "Imprimiendo 0 Documentos" → "Enviando X de Y documento(s)"

Bug: Cuando PS respondía muy rápido (<5ms), los callbacks DONE/FAILED llegaban antes de crear `PSPProgressService`. El modal mostraba "Imprimiendo 0 Documentos" porque `CantidadPendiente` ya era 0 (todos terminaron).

Fix: Cambiar el texto del modal a formato conteo progresivo:

ANTES: "Imprimiendo 0 documento(s)" ← confuso, parece que no envió nada
 AHORA: "Enviando 2 de 4 documento(s)" ← claro, el usuario ve el progreso

Archivos modificados:

Archivo	Cambio
PSPProgressService.cs	Nuevas propiedades <code>CantidadTotal</code> (nunca cambia) y <code>CantidadCompletados</code> (sube reactivamente con <code>OnPropertyChanged</code>)
CustomModalImprimiendoNewView.xaml	Texto cambiado de "Imprimiendo {CantidadPendiente}" a "Enviando {CantidadCompletados} de {CantidadTotal}"

Flujo visual para el usuario:

t=0s Modal: "Enviando 0 de 4 documento(s)" [barra azul 0%]
 t=1s Modal: "Enviando 1 de 4 documento(s)" [barra azul 25%]
 t=2s Modal: "Enviando 3 de 4 documento(s)" [barra azul 75%]
 t=3s Modal: "Enviando 4 de 4 documento(s)" [barra verde 100%] → cierra → éxito

Si hay error:

t=2s Modal: "Enviando 2 de 4 documento(s)" [barra roja 50%] → cierra → modal error